

Advanced Programming 2025

A Comprehensive Analysis of EA FC 26 Player Card Pricing: Predictive Modeling, Feature Attribution and Market Efficiency

Final Project Report

Santiago Tailleferd
santiago.tailleferd@unil.ch
Student ID: 20557377

January 10, 2026

Abstract

The EA FC 26 Ultimate Team transfer market is one of the largest and most complex virtual economies in gaming, characterized by high volatility and heterogeneity in card prices. This study develops a complete machine learning pipeline to predict and interpret card prices in the Ultimate Team transfer market. Two weekly price snapshots were scraped from FUTBIN for a dataset of 801 player cards, enabling temporal validation. Two median-based baselines and four machine learning models were trained on Week 1 prices and tested on Week 2 prices. Model performance was assessed using R^2 , MAE and $RMSE$. Three interpretability analyses were conducted to identify the most important features: OLS coefficients, model-based feature importance and SHAP values. Using predictions from the best-performing model as fair value estimates, potential market mispricing was identified. XGBoost achieved the strongest performance, explaining 95.6% of the variance in card prices, followed by the Random Forest, neural network (MLP) and linear regression. Overall rating and card category emerged as the most impactful features, followed by the number of PlayStyles+, club affiliation and pace. Using a $\pm 20\%$ threshold, 35.1% of player cards were classified as mispriced. Card prices in Ultimate Team are driven by nonlinear interactions between features, leading to persistent deviations between observed prices and model-implied fair values. While the market is largely efficient, machine learning models can capture complex pricing dynamics and identify exploitable inefficiencies.

Keywords: data science, Python, machine learning, XGBoost, SHAP, virtual economy, EA FC 26 Ultimate Team, price prediction, market inefficiencies, temporal validation.

Contents

1	Introduction	3
2	Literature Review / Related Work	4
3	Methodology	4
4	Implementation	5
5	Codebase and Reproducibility	7
6	Results	8
6.1	Exploratory Data Analysis	8
6.2	Model Performance	9
6.3	OLS Analysis, Feature Importance and SHAP Values	9
6.4	Market Inefficiencies	10
7	Discussion	11
7.1	Model Performance	11
7.2	OLS Analysis, Feature Importance and SHAP Values	12
7.3	Market Inefficiencies	12
8	Conclusion and Future Work	13
8.1	Summary	13
8.2	Limitations	13
8.3	Future Work	13
	References	15
A	Additional Figures	16
B	Code Repository	23
B.1	Github Repository	23
B.2	Setup, Environment and Reproducibility	23
B.3	Repository Structure	23
C	AI Usage	25

1 Introduction

EA FC 26 (formerly FIFA until 2023) is one of the most popular sports video games, with its Ultimate Team mode generating \$4.4 billion in revenue in 2024 [2]. Following its release in September 2025, EA FC 26 became the most-downloaded game on the PlayStation Store in the world [6]. Ultimate Team allows players to build squads by acquiring player cards through an in-game transfer market using virtual credits earned by completing objectives, playing matches and winning tournaments. However, it is not possible to directly buy credits with real money.

Every day in Ultimate Team, millions of transactions take place on the transfer market, making it a complex virtual economy. As player cards are only listed and purchased by players (the market is P2P), prices are only determined by supply and demand, which reflects dynamics observed in financial markets. Also, given that each week brings a new event and the release of new players, evolutions, SBCs and packs, the transfer market is highly volatile: prices range from 200 to 9.6 million credits.

Moreover, the Ultimate Team transfer market is very noisy: players often make significant errors when listing a card (e.g., mistyping the price) or purchase cards for emotional reasons such as loyalty to a club or nostalgia for a specific icon, rather than focusing exclusively on in-game features. As a result, prices are not determined only by their numerical features but reflect economic concepts such as scarcity and opportunity cost [7]. This makes price prediction and mispricing analysis meaningful, as trading opportunities may appear when card prices deviate too much from their fundamental value.

At the start of the game, only base cards are available and remain available throughout the whole game: bronze cards (rated 47 to 64), silver cards (rated 65-74), gold cards (rated 75+) and icons/heroes (rated 85+, retired stars). Over time, special cards (rated 75+) such as Team of the Week, Cornerstones and Ratings Reload are released through weekly events and remain available in packs for a week before being replaced by the new event.

Card prices are influenced by many factors, including quantifiable features such as card attributes (e.g., overall rating, pace, number of PlayStyles) and card category (e.g., gold, icons, heroes). However, there are factors that are more difficult to model, such as sentimental preferences, pack availability and card requirements for SBCs and evolutions. Hence, this report addresses the following questions:

What card features influence card prices the most?

Which machine learning model can most accurately predict card prices?

Can prediction models identify market inefficiencies?

To answer these questions, we conducted an analysis of a dataset of 811 player cards observed across two weekly price snapshots to enable temporal validation. We compared six models, including two median-based baselines and four machine learning models: linear regression, Random Forest, XGBoost and a neural network (MLP). In addition, we applied three complementary techniques to identify the most important features: OLS coefficients, model-based feature importance and SHAP values. Finally, using best-performing model's predictions as fair value estimates, we identified mispriced cards that represent potential trading opportunities.

Section 2 reviews the relevant literature; Section 3 describes the methodology; Section 4 discusses the implementation; Section 5 presents the codebase and reproducibility; Section 6 presents the results; Section 7 presents the discussion and Section 8 presents the conclusion.

2 Literature Review / Related Work

Even though EA FC 26 has been available only since the end of September 2025, several studies have examined previous EA FC/FIFA games. Warmayana et al. used XGBoost on Ultimate Team (FIFA 16 to 21) and showed that tree-based models such as Random Forests and XGBoost explain approximately 89% of the variance in card prices. Their XGBoost model outperformed linear regression and identified overall rating as the most important feature that impacts card prices [8]. Moreover, Goldberg conducted an econometric analysis of the Ultimate Team market using panel regression on card features, and found that non-gold cards such as icons, heroes and special cards remain much more expensive even after accounting for higher statistics [4].

In addition to prediction performance, the Ultimate Team transfer market has also been studied as an economic system. Several papers document that the Ultimate Team transfer market is highly volatile and subject to frequent price shocks caused by new event cards, SBCs, evolutions and pack releases [7]. These findings support the use of temporal validation (training on one week and testing on the next) as an appropriate method to assess generalization and reduce overfitting when it comes to predicting card prices. Related work has also used EA FC/FIFA-style features to predict real-world football player market value and transfer fees, achieving R^2 values of 0.86 and 0.90 for Random Forest and XGBoost respectively [1]. Although there are not a large number of studies on EA FC/FIFA, all these results highlight the importance of capturing nonlinear relationships between card features and prices for accurate valuation models.

Finally, across these studies, log prices are commonly used due to the wide range of card prices. Log prices are standard in econometric and machine learning analyses as they stabilize variance and facilitate the interpretation of coefficients and marginal effects [5].

3 Methodology

Unlike financial markets where there are numerous data providers, only a few platforms track card prices in EA FC 26. FUTBIN is an example of an EA FC data provider, offering detailed information regarding market trends, card features and prices, although it does not directly estimate individual card prices [3]. However, since the data was not available to download, a custom data-collection process was implemented.

The sample includes all tradeable, non-goalkeeper player cards with an overall rating of 83 or higher, which resulted in a dataset of 811 cards. To capture the temporal dynamics of the Ultimate Team market, two price snapshots were collected one week apart to enable temporal validation.

Several preprocessing steps were implemented to ensure effective model learning. All numerical variables such as overall rating, pace and number of PlayStyles were standardized using z-score normalization

$$z = \frac{x - \mu}{\sigma},$$

resulting in a standard normal distribution with $\mu = 0$ and $\sigma = 1$. Categorical variables such as nationalities and leagues were one-hot encoded for the top 10 nationalities and the top 8 leagues, representing 70% and 75% of the sample, respectively. The remaining categories were grouped into an “Other” category. However, given the high cardinality of clubs (126), target encoding was used instead of one-hot encoding, so each club was represented by the average price of its cards computed on the training set, in order to avoid creating too many binary columns.

Regarding card positions, both the total number of playable positions per card and position cluster affiliation were considered. Five binary position clusters were defined: center backs, full-backs, midfielders, attacking midfielders and strikers. Cards eligible for multiple positions were assigned to all relevant clusters. Due to the wide price range in the market and the pronounced right skew in prices, card prices were log-transformed before training the models.

Cards were grouped into three categories: gold cards, icons/heroes and special cards, which include various weekly event cards that exhibit similar market behavior. Because players may have multiple card versions (e.g., multiple versions of Vinícius Júnior), each card was treated as a separate observation with distinct features and price. Gender was initially considered as a feature, but was excluded due to perfect collinearity with league affiliation.

Then, two baseline models were constructed to establish reference performance benchmarks. The first baseline predicts Week 2 prices using the median price for each overall rating, while the second baseline improves this approach by using the median price conditional on overall rating and card category.

Four machine learning models were evaluated: linear regression, Random Forest, XGBoost and a neural network (MLP). All models were trained using log-transformed prices and used a consistent set of engineered features to facilitate comparability. Training was conducted on Week 1 prices, while evaluation used Week 2 prices, following a temporal validation framework.

Model performance was assessed using three metrics: coefficient of determination (R^2), mean absolute error (MAE) and root mean squared error ($RMSE$). Both MAE and $RMSE$ are expressed in credits after inverse log-transformation (exponentiation) of predictions and provide measures of prediction errors. However, while MAE treats all deviations equally, $RMSE$ gives more weight to larger errors and is therefore more sensitive to extreme mispricing.

Finally, and to complement predictive performance, several interpretability techniques were applied to explain and identify the key drivers of card prices. First, an ordinary least squares (OLS) regression was estimated to obtain coefficient-based interpretations of marginal effects. Then, model-based feature importance measures and SHAP values (global and local) were computed to capture nonlinear contributions. Both feature importance and SHAP analyses were performed using the best-performing predictive model.

Lastly, by comparing predicted prices obtained from the best-performing predictive model with observed market prices, potentially mispriced cards were identified. A threshold of $\pm 20\%$ was applied to account for the market's high volatility and to avoid classifying small deviations as trading opportunities. However, in the results section, mispricing opportunities were ranked in absolute credit terms rather than in percentage terms, as small relative deviations in high-priced cards may represent larger opportunities than large percentage deviations in low-priced cards.

4 Implementation

Using FUTBIN's filterable list of Ultimate Team cards, the sample was restricted to tradeable, non-goalkeeper cards with an overall rating of 83 or higher. As not all required features were available directly from the list, a multistep data-collection scraper was implemented. First, a URL extractor retrieved the individual FUTBIN page URLs for all eligible cards. A second scraper visited each card page and collected static features such as overall rating and pace. Lastly, card prices, which are time-varying, were handled separately: attributes were first merged with Week 1 prices, and one week later, prices were collected again to obtain Week 2 prices.

During collection, data quality was ensured by removing cards with a price equal to zero. Additionally, eight cards were manually excluded because they were incorrectly listed in the transfer market as belonging to their former club for a short period. After cleaning, 802 cards were retained for exploratory data analysis (Week 1 only), while 801 cards with non-zero prices in both weeks were used for predictive modeling.

Given the high cardinality of clubs (126), target encoding was used instead of one-hot encoding, with a smoothing parameter of 10 to prevent overfitting on rare clubs, as shown in Listing 1.

```

1 def create_club_encoding_map(
2     df: pd.DataFrame, target_col: str, smoothing: int = 10
3 ) -> Dict[str, float]:
4     """
5     Create club target encoding map from training data only.
6     Use smoothing (10) to prevent overfitting on rare clubs.
7     """
8     global_mean = df[target_col].mean()
9     club_stats = df.groupby("club")[target_col].agg(["mean", "count"])
10
11     # Apply smoothing formula to prevent overfitting on rare clubs
12     club_stats["smoothed_mean"] = (
13         club_stats["count"] * club_stats["mean"] + smoothing * global_mean
14     ) / (club_stats["count"] + smoothing)
15
16     # Create encoding map for seen and unseen clubs in W2
17     club_encoding_map = club_stats["smoothed_mean"].to_dict()
18     club_encoding_map["__global_mean__"] = global_mean
19
20     return club_encoding_map

```

Listing 1: Smoothed target encoding for club (fit on Week 1 only)

To prevent data leakage, all preprocessing steps were fitted on Week 1 data only. The same parameters were then applied directly to Week 2, without any modification to the test set, as shown in Listing 2.

```

1 # Prepare train features (Week 1)
2 x_train, y_train_log, scaler, club_map, feature_names = prepare_features(
3     df,
4     target_col="price_w1",
5     scaler=None,
6     club_encoding_map=None,
7     smoothing=10,
8     feature_names=None,
9 )
10
11 # Prepare test features (Week 2)
12 x_test, y_test_log, _, _, _ = prepare_features(
13     df,
14     target_col="price_w2",
15     scaler=scaler,
16     club_encoding_map=club_map,
17     smoothing=10,
18     feature_names=feature_names,
19 )

```

Listing 2: Temporal validation: preprocessing fitted on Week 1 only

Models were evaluated using temporal validation. Cross-validation was applied only to Week 1 data to check model stability and overfitting, while Week 2 was kept as a fully out-of-sample test set to replicate real trading conditions.

Machine learning models were implemented with the following principal configurations:

- **Random Forest:** 500 trees, maximum depth of 20, minimum samples per split of 5
- **XGBoost:** 1,000 trees, learning rate of 0.03, maximum depth of 4
- **neural network (MLP):** four-layer architecture ($128 \rightarrow 64 \rightarrow 32 \rightarrow 1$) with dropout regularization (30% decreasing to 20%). Training used early stopping and stabilized after 57 epochs.

Using the best-performing model, market inefficiencies were detected by comparing predicted prices with actual market prices. Listing 3 shows the classification logic: players were labeled as underpriced or overpriced if the relative deviation exceeded a $\pm 20\%$ threshold.

```
1 # Calculate price difference using XGBoost predictions (best model)
2 df["price_diff"] = df["pred_price_w2"] - df["price_w2"]
3 df["price_diff_pct"] = (df["price_diff"] / df["price_w2"]) * 100
4 df["abs_price_diff"] = df["price_diff"].abs()
5
6 # Classify players (threshold: 20% relative difference)
7 threshold = 20
8 df["status"] = "Fair Value"
9 df.loc[df["price_diff_pct"] > threshold, "status"] = "Underpriced"
10 df.loc[df["price_diff_pct"] < -threshold, "status"] = "Overpriced"
```

Listing 3: Market inefficiency detection using XGBoost predictions

5 Codebase and Reproducibility

The project structure is modular and clearly organized. Both the raw and processed data are stored in the `data/` directory, while the figures, tables and price predictions are saved in the `results/` directory. Most importantly, the `src/` directory is divided into four folders: `scrapers/`, `preprocessing/`, `models/` and `analysis/`. This organization makes it easy to modify or extend the project (e.g., adding new models or feature engineering steps) without having to change the rest of the code. The complete repository tree is reported in Appendix B1

Except for the scrapers which must be run separately, the entire code can be run directly by executing `main.py`. Running this file reproduces all results presented in the report and repeated executions will provide identical results due to the fixed random seed (42) across all models. To ensure reproducibility, the project was developed within a Conda environment using Python 3.10. A `requirements.txt` file is included in the repository to allow the software environment to be replicated on a different machine. The following libraries were used:

- **Web scraping:** Requests and BeautifulSoup4 were used to extract data from FUTBIN (HTML).
- **Data manipulation:** Pandas and NumPy were used for data cleaning, merging and dataset operations.
- **Machine learning:** Scikit-learn was used for linear regression and Random Forest, XGBoost was used for XGBoost model and TensorFlow was used for the neural network.
- **Interpretability:** SHAP was used to calculate SHAP values and explain model outputs.
- **Visualization:** Matplotlib and Seaborn were used for exploratory data analysis (EDA) and plotting results.

Code maintenance followed standard software development practices. All changes were tracked with Git, which made it easier to work incrementally, test new ideas and revert changes if necessary. Code formatting was standardized using Black to maintain a consistent style and improve overall readability.

The project also includes a dedicated `tests/` directory with five automated unit tests. These tests check data integrity, feature engineering logic, evaluation metrics and training stability for the two best-performing models. In case future modifications or improvements are made in this project, these tests would ensure that the code behaves consistently over time.

6 Results

6.1 Exploratory Data Analysis

Exploratory data analysis indicates high heterogeneity in card prices within the Ultimate Team transfer market. Figure 1 shows that card prices vary substantially by category: gold cards have a median price of 3,300 credits, special cards 18,635 credits and icons/heroes 215,000 credits. Correlation analysis (Figure A2) reveals that overall rating has the strongest linear relationship with log price ($r = 0.55$), followed by the number of PlayStyles+ ($r = 0.28$) and skill moves ($r = 0.27$). The positive relationship between overall rating and log price is illustrated in Figure A3. In addition, Figure A4 shows that features such as pace, dribbling, skill moves and weak foot are also positively correlated with log price, as expensive cards tend to have stronger values for these features. Finally, Figure A5 shows that when cards are segmented by cluster position, significant differences are also observed: center backs have a median price of 12,750 credits, while strikers have a median price of 18,125 credits, corresponding to a difference of more than 40%.

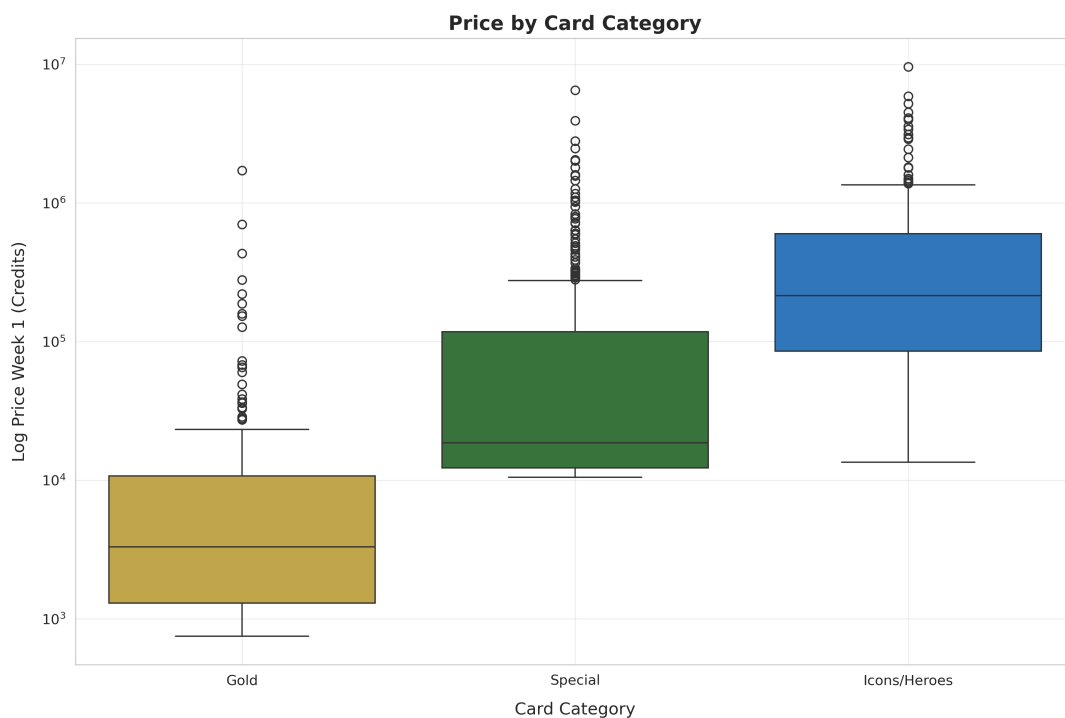


Figure 1: Price by Card Category (Credits).

6.2 Model Performance

Regarding both median-based baselines, baseline 2 (taking into account both overall rating and card category) was considered the benchmark, with an R^2 of 0.633, MAE of 129,361 credits and $RMSE$ of 397,507 credits. Baseline 1 was the worst with an R^2 of 0.608.

Among the four machine learning models, XGBoost achieved the strongest performance with an R^2 of 0.956 and the lowest error metrics: MAE of 42,997 credits and $RMSE$ of 137,993 credits. This corresponds to an improvement of 32.3 percentage points in R^2 compared to the benchmark (baseline 2). Random Forest ranked second with an R^2 of 0.86 and higher error metrics than XGBoost. The neural network (MLP) ranked third with an R^2 of 0.669, slightly above the benchmark. Lastly, linear regression was the worst among the four machine learning models with an R^2 of 0.631, slightly below the benchmark. The full comparison can be found below in Table 1 and Figure A6.

Table 1: Model Performance Metrics (ranked by R^2)

Model	R^2	RMSE	MAE	R^2 improvement over benchmark
XGBoost	0.956	137,993	42,997	32.3%
Random Forest	0.860	245,831	63,961	22.7%
Neural network (MLP)	0.669	377,422	92,889	3.6%
Baseline 2 (benchmark)	0.633	397,507	129,361	0.0%
Linear regression	0.631	398,273	116,050	(0.2%)
Baseline 1	0.608	410,874	142,364	(2.5%)

6.3 OLS Analysis, Feature Importance and SHAP Values

The OLS regression (Figure A7 and Figure A8) indicates that 32 out of 41 features are significant (with $p < 0.05$). Card category variables such as special cards (2.86) and icons/heroes (2.61) have the largest positive effects on card prices, while the fullback position cluster (-0.25) and dribbling (-0.14) have the most negative impact. Residual diagnostics (Figure A9) show an approximately symmetric distribution, with some deviations in the tails.

Feature importance derived from XGBoost shows that only the top three features account for more than 60% of total importance, while the 80% threshold is surpassed by the top eight features (Figure A10). Consistent with the OLS results, card category variables have the largest impact on card prices (36.3% of total importance), followed by overall rating and the number of PlayStyles+ (21% and 14% of total importance respectively).

Finally, global SHAP values (Figure 2) corroborate the XGBoost feature importance ranking, with overall rating, card category variables and pace having the largest impact. At the local level and as an example, Ronaldo Nazário, the most expensive player in the dataset, was examined (Figure A11). With an actual price of 9,390,000 credits and a predicted price of 8,565,180 credits (8.8% error), SHAP values break down this prediction. Starting from a base log price of 10.21, they show how each feature contributes positively or negatively to the final prediction. For example, Ronaldo Nazário's overall rating increases the predicted log price by 1.81.

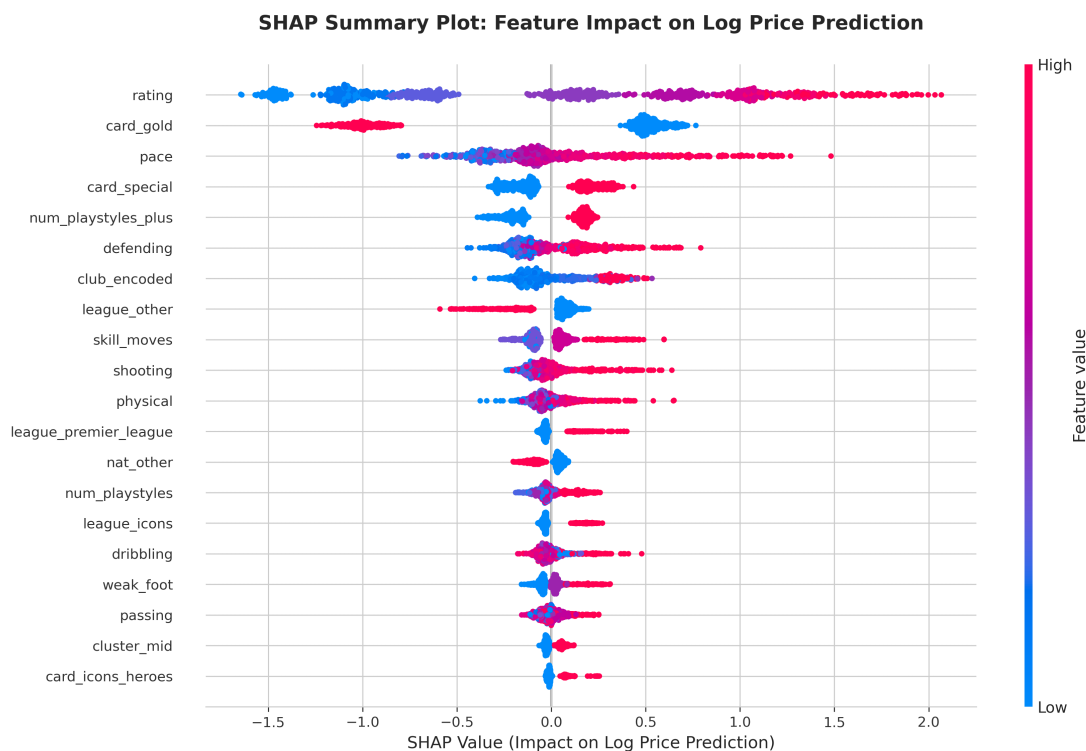


Figure 2: SHAP Summary Plot: Feature Impact on Log Price Prediction.

6.4 Market Inefficiencies

Using XGBoost predictions as fair value estimates, market inefficiencies were detected by comparing predicted with actual prices. Cards were classified as underpriced or overpriced if the relative difference exceeded $\pm 20\%$. As shown in Figure 3, most observations lie close to the diagonal, indicating fair valuation, while deviations below and above the diagonal correspond to underpriced and overpriced cards.

Overall, 64.9% of cards fall within the $\pm 20\%$ band and are considered fairly valued. However, 25.5% of cards are underpriced, while only 9.6% are overpriced. Figure A12 presents the top fifteen underpriced and overpriced cards ranked by absolute credit deviations: Johan Cruyff, Ousmane Dembélé and Vinícius Jr. were classified as the top three most underpriced cards while Ronaldinho, Zlatan Ibrahimović and Virgil van Dijk were classified as the three most overpriced cards in the sample.

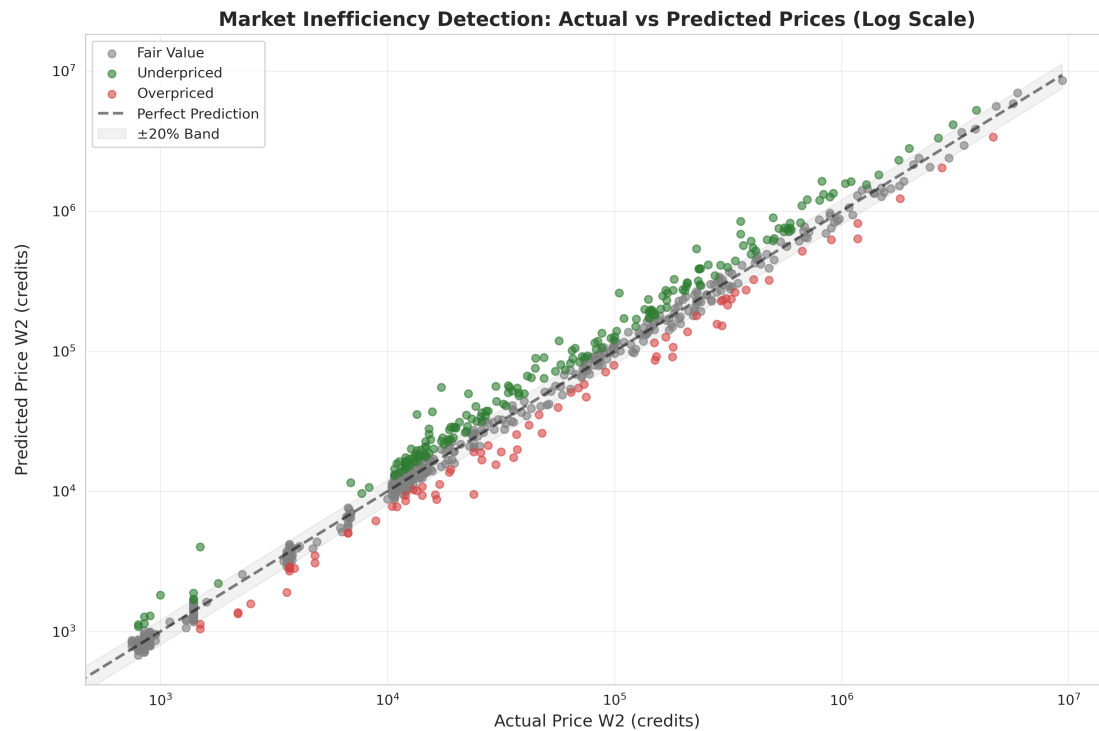


Figure 3: Market Inefficiency Detection: Actual vs Predicted Prices.

7 Discussion

7.1 Model Performance

Overall, the comparison shows that nonlinear models outperform linear models and simple baselines when predicting player prices in the Ultimate Team transfer market.

The strong absolute and relative performance of XGBoost ($R^2 = 0.956$) suggests that there are nonlinear relationships between card features and card prices that linear regression cannot capture: high pace matters more for strikers than for midfielders, which can only be modeled effectively with tree-based models such as XGBoost. This result is consistent with Warmayana et al. who also report stronger performance for XGBoost on the FIFA 16-21 Ultimate Team [8]. The slightly higher R^2 could be due to the more extensive feature engineering such as position clustering and target encoding for clubs which allowed the model to have input features more complete. Moreover, despite its strong performance, the model did not overfit as the train-test gap was small (Train $R^2 = 0.971$, Test $R^2 = 0.956$). 10-fold cross-validation also confirmed acceptable stability, resulting in an R^2 of 0.921 ± 0.043 and achieving the lowest variance among all models.

Random Forest achieved strong performance too ($R^2 = 0.86$), but was outperformed by XGBoost. This is again consistent with Warmayana et al. [8] who showed that tree-based models perform well in predicting card prices. However, it also shows the difference between both tree-based models: while XGBoost builds trees sequentially to correct previous errors, Random Forest builds them independently and averages them. The sequential approach appears to be better for this project.

The weaker performance of the neural network ($R^2 = 0.669$) compared to XGBoost and Random Forest is not surprising as the sample of 801 cards was relatively small. Also, MLPs are more complex models that ideally require more observations than the rest of the models. The model stabilized without overfitting as training stopped at epoch 57 out of 300. However it lacked observations and its performance was close to linear regression and benchmark.

As expected, linear regression ($R^2 = 0.631$) performed worst among the four machine learning models. However, it is surprising that it even underperformed compared to the benchmark ($R^2 = 0.633$) which only takes the average price of cards given their overall rating and category. This supports the conclusion that the relationship between card features and card prices is highly nonlinear. It also confirms that overall rating and card category are the two most important features that drive card prices, as they both explain more than 60% of the variance in card prices according to the benchmark.

7.2 OLS Analysis, Feature Importance and SHAP Values

The combination of OLS coefficients, XGBoost feature importance and SHAP values provides a consistent picture: card category variables and overall rating are the two most influential features for both linear and nonlinear models. This is consistent with Goldberg's observation that non-gold cards such as icons, heroes and special cards remain much more expensive even after accounting for higher statistics [4].

Beyond these primary features, there are secondary yet important features such as the number of PlayStyles+, club affiliation and pace which also contribute to the price variation. However, their effects are marginal rather than decisive. This shows that only a few features contribute to card prices while the majority of them only have a negligible impact.

It is interesting to note that on average, strikers cost almost three times as much as fullbacks and attacking midfielders cost twice as much as fullbacks. Although defending is as important as attacking in the game, this large difference is mainly driven by the fact that it is more fun to play with strikers and attacking midfielders, but also that players prefer to allocate more money to attacking positions rather than defending positions. This confirms that card prices are not only driven by numerical features but also by how players are used and valued in-game.

7.3 Market Inefficiencies

While 64.9% of cards are priced within $\pm 20\%$ of predicted fair value, the distribution of mispricing is asymmetric: 25.5% are underpriced while only 9.6% are overpriced. This indicates that the market is reasonably efficient overall, even if inefficiencies exist given its P2P market structure. When an overpriced card fails to sell, it sends a clear signal to the seller to lower the price until it eventually falls within the fair value threshold. However, there are no equivalent signals for players to detect underpriced cards: buyers simply purchase a card at the listed price without signaling that it was too cheap, and sellers have no way of knowing their card could be more expensive. As a result, overpricing corrects quickly while underpricing tends to persist.

Extreme deviations from the actual price correspond to players with strong emotional and nostalgic appeal such as Ronaldinho, whose predicted value is 1.26 million credits below the actual price. Players are willing to pay a premium for legends and players they support in real life. Sentiment-driven demand is very hard to model as it represents an intangible factor that extends beyond observable card features. This confirms that card prices in Ultimate Team are more complex than a simple attribute-based analysis.

8 Conclusion and Future Work

8.1 Summary

This project developed a complete machine learning pipeline to predict and interpret card prices in the EA FC 26 Ultimate Team transfer market. Using scrapers, extensive feature engineering, two baseline models, four machine learning models and three interpretability analyses, the project demonstrated that machine learning can accurately predict card prices and identify market inefficiencies.

Among the four machine learning models implemented, XGBoost achieved the best performance explaining 95.6% of the variance in card prices, followed by the Random Forest, neural network (MLP) and linear regression. XGBoost outperformed the benchmark by 32.3 percentage points in R^2 . In contrast, linear regression performed worse than the benchmark, showing that the relationship between card features and card prices is not linear.

Three interpretability techniques were implemented: OLS analysis, feature importance and SHAP values (last two derived from the XGBoost model). Overall rating and card category were the two variables that had the biggest impact on card prices, followed by the number of PlayStyles+, club affiliation and pace.

Finally, taking XGBoost predictions as fair value estimates and a $\pm 20\%$ margin of error, the analysis identified potential market inefficiencies. 64.9% of players were classified as fairly priced, while 35.1% were classified as mispriced, suggesting potential trading opportunities.

8.2 Limitations

This project has several limitations. First, the sample excludes goalkeepers as their features differ from those of field players and they only represent a small portion of EA FC 26 cards. The sample was restricted to cards rated 83 or higher as lower-rated cards rarely exceed their minimum prices. While two weekly snapshots are enough to demonstrate temporal validation, they are not sufficient to capture the game's long-term dynamics or the gradual depreciation of the game cycle. The project also relies on two discrete price snapshots rather than continuous prices.

Regarding feature engineering, some simplifications were made to avoid overdimensioning. Only top nationalities and top leagues were one-hot encoded while clubs were target-encoded. Only the number of PlayStyles and PlayStyles+ was used instead of encoding individual indicators. Card categories were divided into three types (gold, special, icons/heroes) rather than encoding each special card variant separately. However, for the scope of this project, these choices were appropriate and sufficient.

Finally, some variables could not be included due to data availability and collection constraints, such as player popularity rankings, SBC and evolution requirements, pack availability and real-world performance indicators. Since the analysis ends at Week 2, it is also not possible to determine whether the identified inefficiencies in Week 2 generated profits in later weeks.

8.3 Future Work

Several directions could extend this work. First, a real-time system with automated periodic scraping and retraining would enable the models to adapt their price predictions more quickly and accurately. Ideally, modeling prices as a continuous time series would capture dynamics and signals that cannot be detected using weekly or even daily snapshots.

Feature engineering could also be improved by including meta ratings derived directly from the game (e.g., number of games played or goals scored by a card), real-world performance data (e.g., current form or injuries) and sentiment indicators from social media such as X (e.g., player mention frequency) or Instagram (e.g., number of followers). Furthermore, adding pack availability, SBC and evolution requirements would help capture price drivers that are not reflected in card features alone.

References

- [1] Ali Al-Asadi and Kadri Tasdemir. “Predicting the Value of Football Players Using FIFA Video Game Data and Machine Learning”. In: *Procedia Computer Science* (2021). Machine learning models applied to Ultimate Team player valuation.
- [2] Electronic Arts Inc. *Electronic Arts Annual Report 2024*. <https://ir.ea.com/financials/annual-reports-and-proxy-information/default.aspx>. Accessed for EA FC and Ultimate Team revenues. 2024.
- [3] FUTBIN. *FUTBIN — EA FC Ultimate Team Database*. <https://www.futbin.com>. Database used to scrape EA FC 26 Ultimate Team player card attributes and prices. 2025.
- [4] Adam Goldberg. “An Econometric Analysis of a \$1.4 Billion In-Game Market”. In: *Available as Working Paper* (2019). Econometric study of Ultimate Team transfer market.
- [5] Z. Huang et al. “Explainable Machine Learning for Football Player Market Value Prediction”. In: *arXiv preprint* (2023). Use of SHAP values for model interpretability.
- [6] PlayStation Blog. *PlayStation Store: September 2025 Top Downloads*. <https://blog.playstation.com/2025/10/02/playstation-store-september-2025s-top-downloads/>. Official PlayStation Store download statistics. 2025.
- [7] Brian Sheehan and Robert Marks. “Economics Lessons in FIFA Ultimate Team”. In: *Journal of Sports Economics* (2018). Analysis of virtual markets and pricing mechanisms in Ultimate Team.
- [8] I. M. Warmayana et al. “Predicting FIFA Ultimate Team Player Prices Using Machine Learning”. In: *International Journal of Research in Marketing* (2021). Application of Random Forest and XGBoost to Ultimate Team price prediction.

A Additional Figures

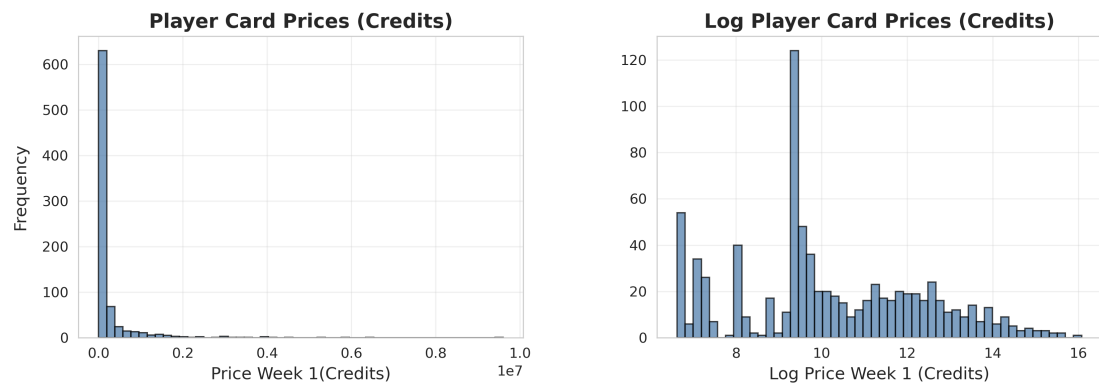


Figure A1: Distribution of Player Card Prices in Credits and Log Scale.

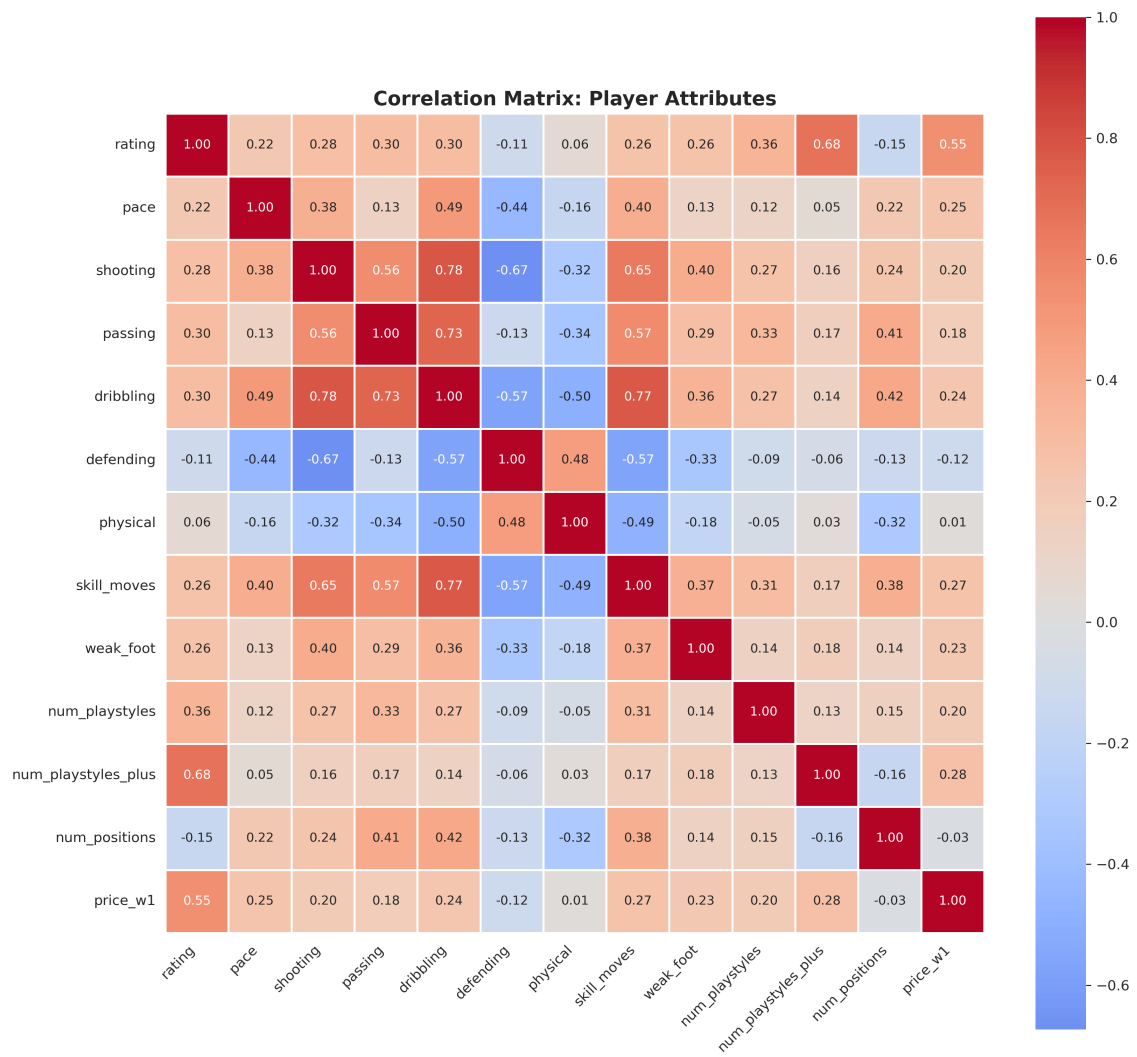


Figure A2: Correlation Matrix of Player Attributes and Prices.

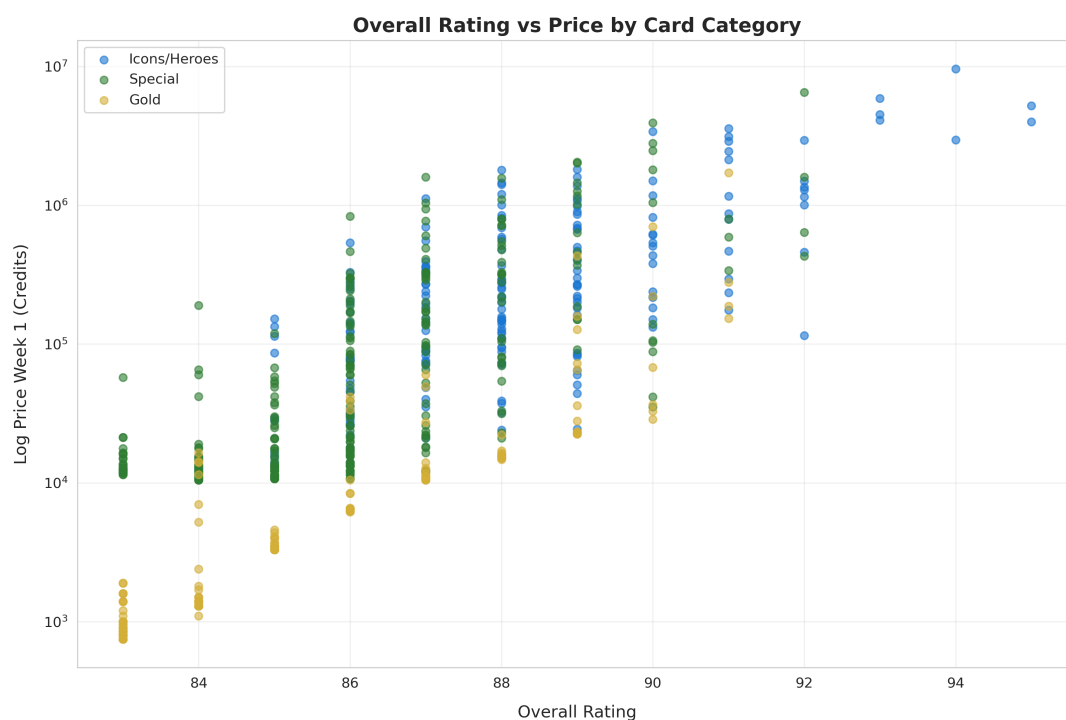


Figure A3: Overall Rating vs Log Price by Card Category.



Figure A4: Relationships Between Key Attributes and Log Price.

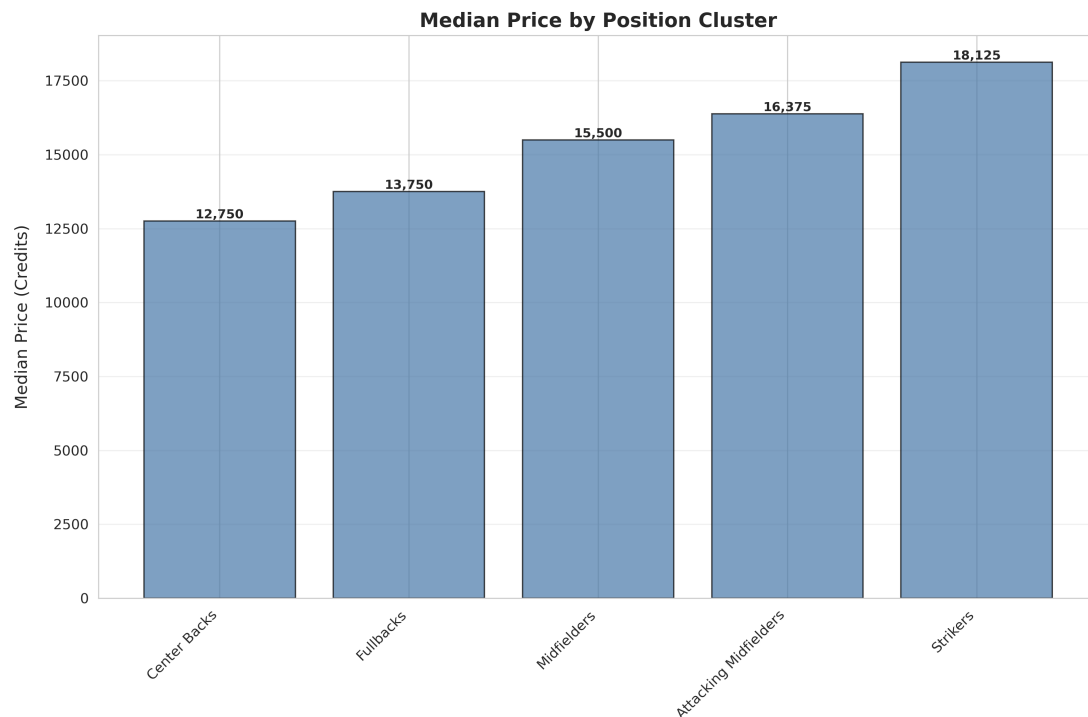
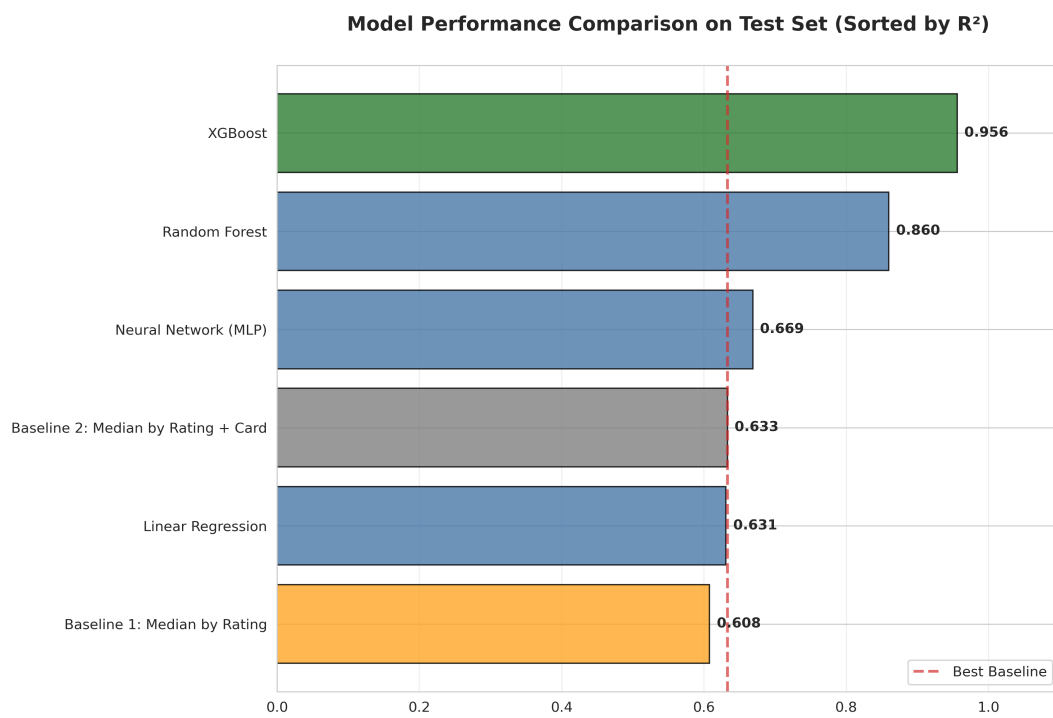


Figure A5: Median Log Price by Position Cluster.

Figure A6: Comparison of Model Performance (R^2).

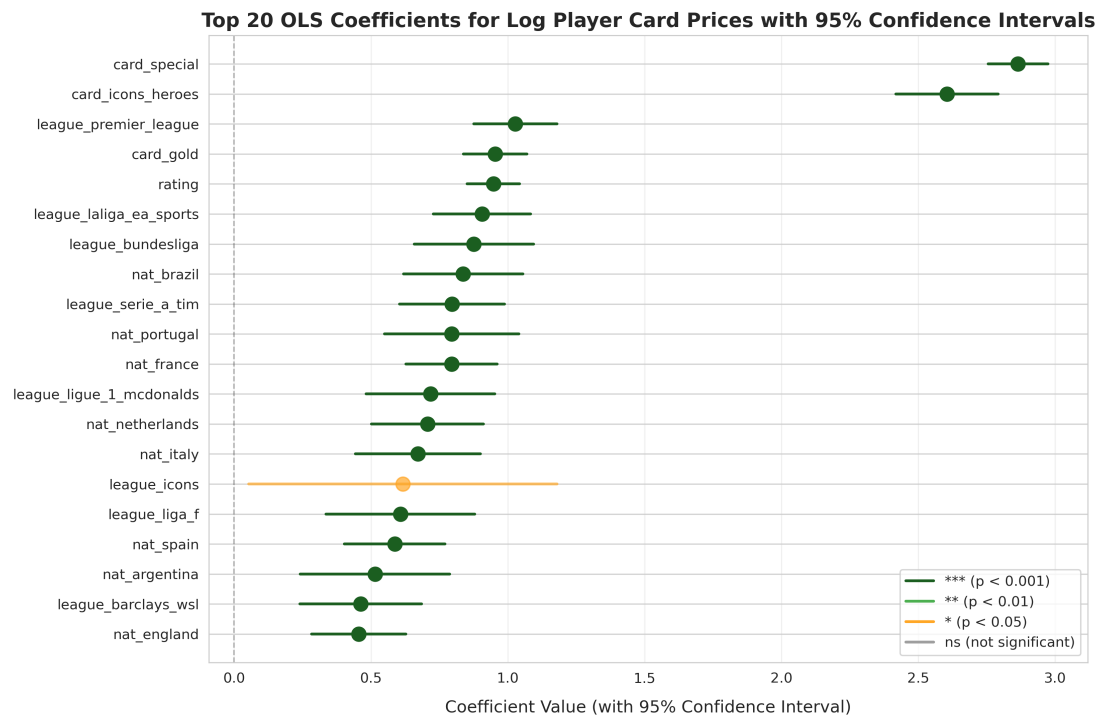


Figure A7: Top OLS Coefficients for Log Price with 95% Confidence Intervals.

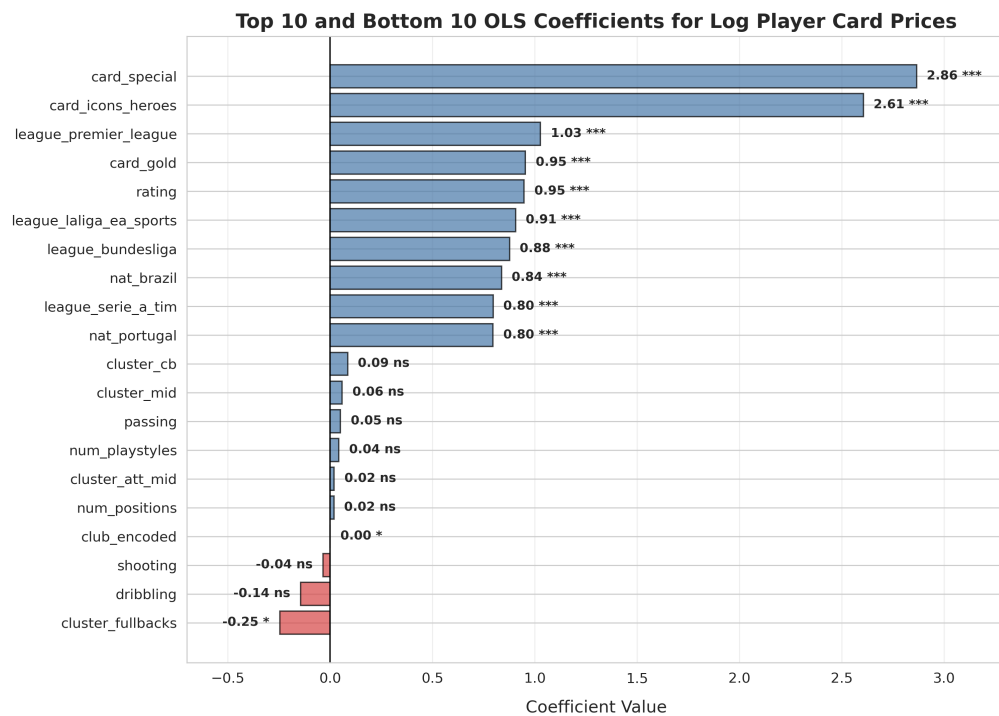


Figure A8: Top and Bottom OLS Coefficients for Log Price Prediction.

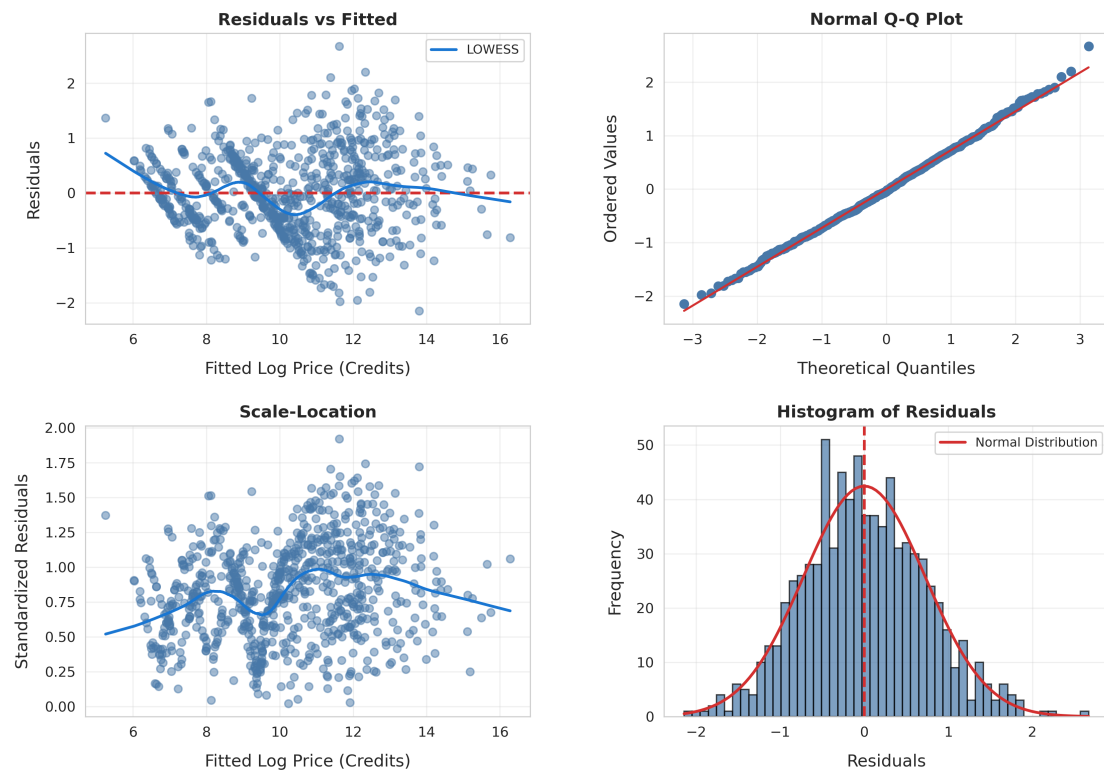


Figure A9: OLS Residual Diagnostics: Linearity, Normality and Homoscedasticity.

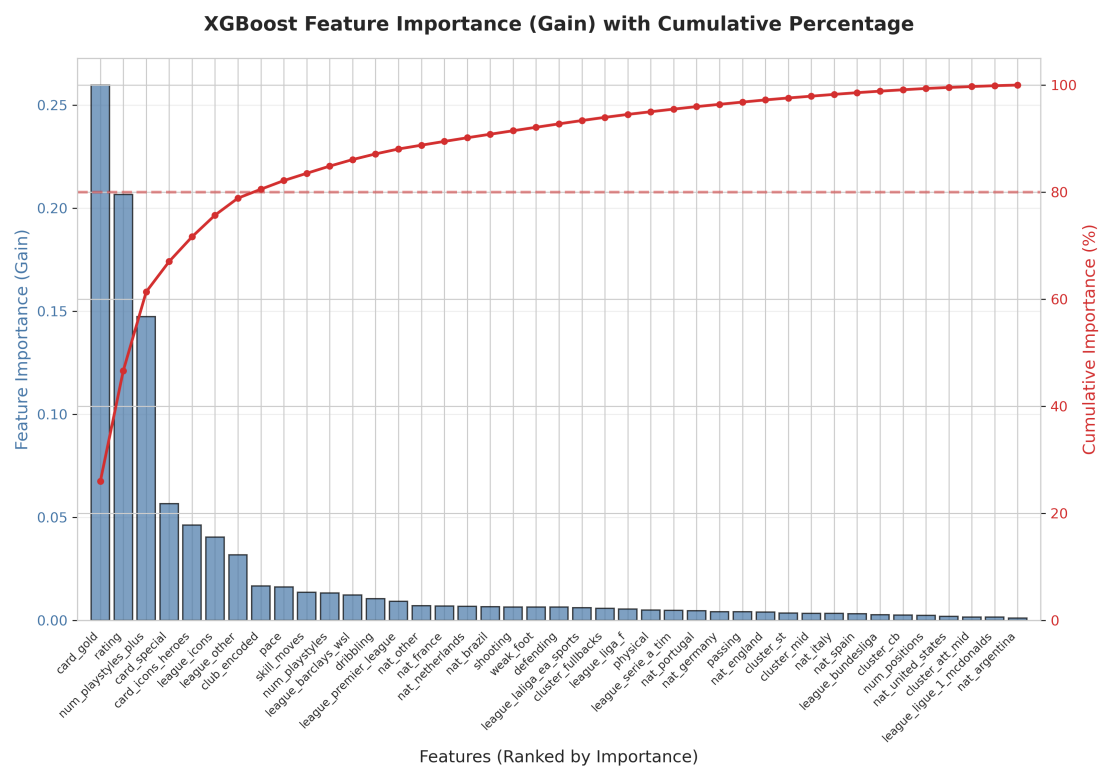


Figure A10: XGBoost Feature Importance (Gain) with Cumulative Contribution.

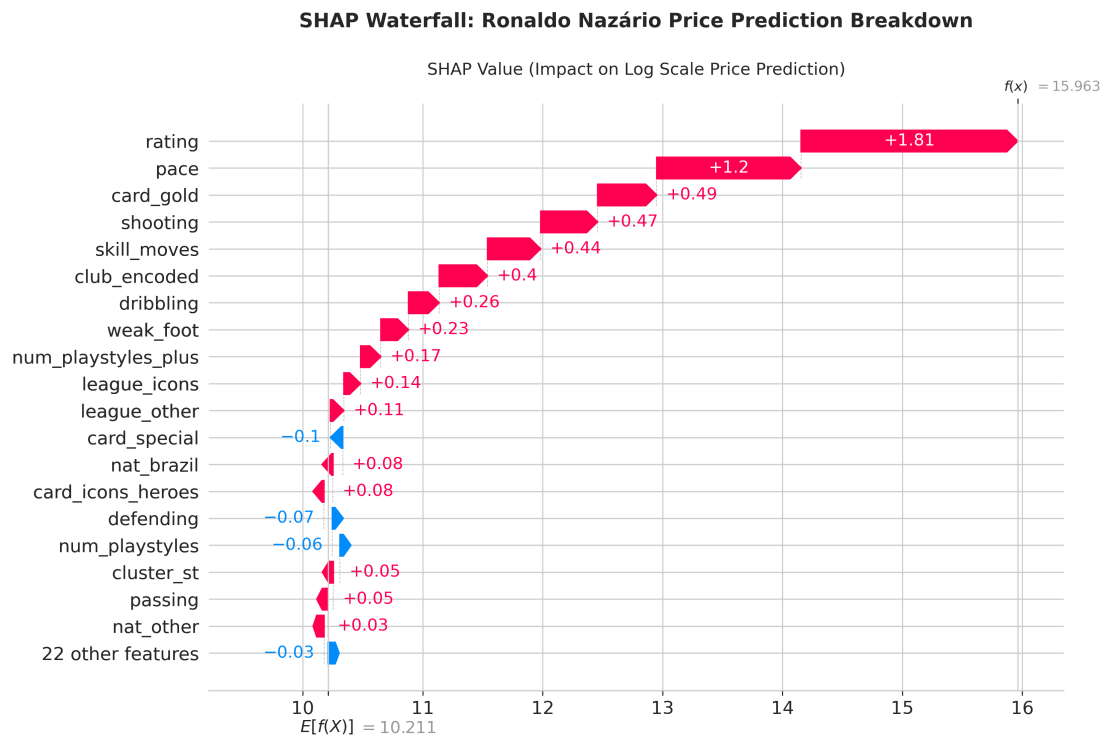


Figure A11: SHAP Waterfall Decomposition for an Individual Player Prediction.

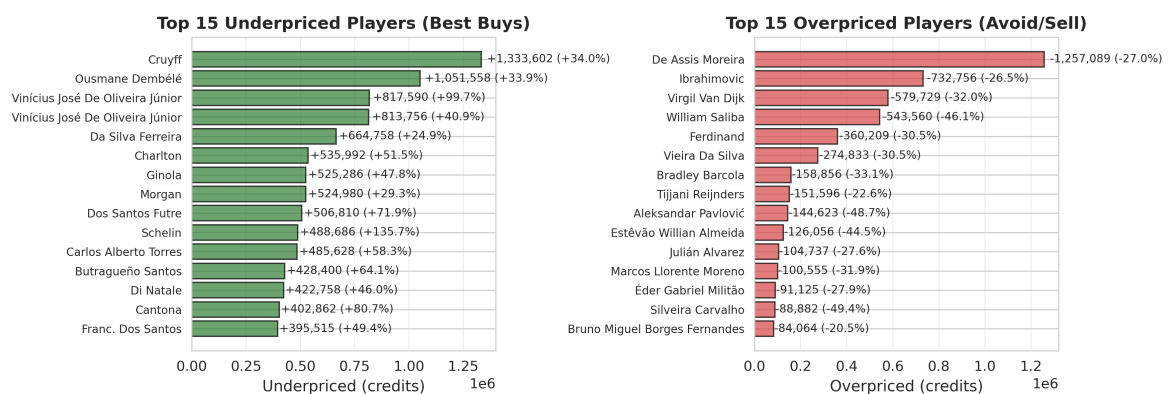


Figure A12: Top Underpriced and Overpriced Players Identified by the Model.

B Code Repository

B.1 Github Repository

GitHub Repository: https://github.com/santitaille/Capstone_Project_ST.git

B.2 Setup, Environment and Reproducibility

The project was developed and tested using Python 3.10 within a Conda environment, using VS Code and the Nuvolos platform. All models use a fixed random seed (42) to ensure reproducibility.

Note: Data scrapers are included as they are part of the project but do not need to be executed to reproduce the results. Otherwise, temporal validation would not be possible. Executing `main.py` generates all figures, tables and predictions presented in the report.

1. Clone the repository

```
git clone https://github.com/santitaille/Capstone_Project_ST.git
cd Capstone_Project_ST
```

2. Create a Conda environment (Python 3.10)

```
conda create -n eaafc_dsap python=3.10 -y
conda activate eaafc_dsap
```

3. Install dependencies

3.1. Unpinned environment

```
pip install -r requirements.txt
```

3.2. Fully pinned environment

```
pip install -r requirements-pinned.txt
```

4. Run the full pipeline

```
python main.py
```

B.3 Repository Structure

The project structure is modular and clearly organized. The complete repository structure is presented below:

- **data/**: Raw, intermediate and fully merged datasets
- **results/**: All generated outputs, including 16 figures, 4 prediction files (one per model) and 5 tables (evaluation metrics and interpretability results)
- **src/**: Source code
 - **scrapers/**: Data collection from FUTBIN
 - * `attributes_scraper.py`
 - * `price_scraper.py`
 - * `url_scraper.py`
 - **preprocessing/**: Data merging and feature engineering
 - * `data_merger.py`
 - * `feature_engineering.py`
 - **models/**: Model implementations
 - * `baseline_models.py`
 - * `linear_regression.py`
 - * `neural_network.py`
 - * `random_forest.py`
 - * `xgboost_model.py`
 - **analysis/**: Evaluation and interpretability
 - * `eda_analysis.py`
 - * `market_inefficiencies.py`
 - * `model_comparison.py`
 - * `ols_coefficient_analysis.py`
 - * `ols_visualizations.py`
 - * `shap_analysis.py`
 - * `xgboost_feature_importance.py`
- **tests/**: 5 automated unit tests
- **main.py**: Single entry point reproducing all results
- **ST_project_report.pdf**: Final project report
- **PROPOSAL.md**: Approved project proposal
- **AI_USAGE.md**: AI usage declaration
- **README.md**: Setup instructions and project overview
- **requirements.txt**: Main Python dependencies (unpinned)
- **requirements-pinned.txt**: Fully pinned environment

Figure B1: Repository structure of the project.

C AI Usage

This project was developed by Santiago Tailleferd. However, the following AI and external tools were used for specific cases. In any case these tools replaced my own thinking or independently generated the code, report or video script.

1. Code Development

Tools used: Claude, ChatGPT, YouTube tutorials and library documentation

These tools were only used to:

- Debug Python code and understand error messages
- Learn and understand new libraries (BeautifulSoup for example)
- Improve plot formatting and clarity
- Converting hard-coded paths to relative paths
- Improve code structure

2. Report and README

Tools used: ChatGPT and Grammarly

These tools were only used to:

- Format text in LaTeX and Markdown
- Correct English grammar and improve clarity

3. Video Presentation

Tools used: ChatGPT and Grammarly

These tools were only used to:

- Ensure I was not missing important information in my script
- Correct English grammar and improve clarity